

Smart Nutrition Assistant

Full Stack Web Development

Project Documentation Report

Technology Stack: React.js · Node.js · Express.js · MongoDB

Architecture: MVC Pattern (REST API + SPA Frontend)

Academic Year: 2025 – 2026

Submitted by:

Ch.Pravalika(AP22110011095)

Krishna Shree GK(AP23110010279)

Nishita Jaya Bhimavarapu(AP23110010048)

Nikhita Jaya Bhimavarapu(AP23110010036)

K.V.L Kinnera(AP23110010071)

1. Introduction

Keeping track of what you eat sounds simple — until you actually try to do it consistently. Between busy schedules, unfamiliar food labels, and the sheer variety of Indian meals that rarely appear in standard nutrition apps, most people give up within a week. That is exactly the gap this project aims to close.

Nutrition Assistant is a full-stack web application built to make daily food tracking straightforward and genuinely useful. It lets users log their meals, see a real-time breakdown of calories and macronutrients, and get personalised suggestions based on their health goal — all in one place. On the backend, it runs on Node.js and Express with a MongoDB database; on the frontend, it uses React with Tailwind CSS for a clean, responsive interface.

1.1 Why We Built This

Most nutrition apps in the market either lack Indian food options or are too complicated for everyday use. We wanted something that felt familiar and practical — an app where a student or working professional could log 'Masala Dosa' or 'Dal Tadka' without having to manually enter every nutrient value from scratch. We also wanted the app to think a little — to notice when your protein is low or when you are consistently hitting your calorie goal, and actually tell you about it.

1.2 What the Project Sets Out to Do

- Let users create an account, pick a health goal, and start tracking meals right away.
- Offer a ready-to-use food catalogue with common Indian meals, pre-loaded with accurate nutritional data.
- Show a live breakdown of calories, protein, carbs, and fats after every meal is logged.
- Give users a Smart Assistant Hub on their dashboard — daily analysis, personalised food suggestions, and a weekly habit score.
- Keep all data secure with JWT-based authentication and bcrypt password hashing.
- Give admins full control to add, edit, or remove food items from the global catalogue.

2. Scenario-Based Case Study

2.1 Meet Alex

Alex is a 26-year-old college student who recently started going to the gym. He wants to build muscle but has no idea how much protein he is actually eating each day. He has tried a couple of apps before but found them either too generic or missing the Indian foods he eats at home. A friend suggests he try Nutrition Assistant.

2.2 Alex's Experience With the App

1. **Signing Up and Setting a Goal** — Alex registers in under a minute. He enters his weight, picks 'Muscle Gain' as his goal, and the app automatically sets his daily calorie target to 2,400 kcal. He does not have to figure that number out himself.
2. **Logging His First Meal** — For breakfast he searches 'Idli' and finds it immediately. He adds three pieces and the app shows him 180 kcal, 9 g protein, 36 g carbs, and 1.5 g fat — instantly.
3. **Getting a Nudge from the Dashboard** — By 3 PM his Smart Assistant Hub shows a warning: his protein is running low for the day. It suggests adding Chicken Breast or Paneer. He makes a mental note to grab some paneer bhurji later.
4. **Checking His Weekly Progress** — At the end of the week, the dashboard shows he logged meals 5 out of 7 days — 71% consistency. He can see exactly which days he missed, which motivates him to do better next week.
5. **Updating His Profile** — A month later Alex has gained some weight and wants to shift to maintenance mode. He updates his goal to 'Maintain' and his calorie target adjusts to 2,000 kcal automatically.

3. System Requirements

Before setting up the project, make sure your development environment meets the following requirements. The app was built and tested with these versions, so sticking close to them will save you debugging time.

3.1 Software Requirements

Tool	What It Does
Operating System	Windows 10/11, macOS, or any Linux distribution
Node.js	v16 or above — runs the backend server and manages packages
npm	v8 or above — installs all React and Express dependencies
React.js	v18 — builds the frontend interface
Vite	Fast dev server with hot module replacement for React
Express.js	v5 — handles all API routing on the backend
MongoDB	Stores all user, food, and meal data
Mongoose	v9 — lets us define schemas and interact with MongoDB cleanly
Tailwind CSS	Makes the UI look good without writing custom CSS from scratch
Chrome / Firefox	Latest version — for testing and using the app
VS Code	Our preferred editor — has great support for JS and React
Postman	Used during development to test every API endpoint

3.2 Hardware Requirements

What You Need	Details
Processor	Intel Core i5 (8th Gen+) or AMD Ryzen 5 — anything faster is fine
RAM	8 GB minimum; 16 GB makes running the dev server, database, and browser simultaneously much smoother
Storage	At least 1 GB free for packages, the database, and project files
Display	1366 × 768 or higher so the UI renders properly
Network	Needed during setup to install packages via npm

3.3 Backend NPM Packages

These are the packages the backend depends on. They all install automatically when you run `npm install`.

Package	Version and Purpose
<code>express</code>	v5.2.1 — the web framework everything is built on
<code>mongoose</code>	v9.4.1 — connects to MongoDB and manages our data schemas
<code>jsonwebtoken</code>	v9.0.3 — creates and verifies JWT tokens for authentication
<code>bcryptjs</code>	v3.0.3 — hashes passwords before saving them to the database
<code>dotenv</code>	v17 — loads environment variables from the <code>.env</code> file
<code>cors</code>	v2.8.6 — lets the React frontend talk to the Express backend
<code>express-validator</code>	v7.3.2 — validates incoming request data before it hits the database
<code>nodemon (dev)</code>	v3.1.14 — restarts the server automatically when you save a file

4. Project Architecture

The app is split cleanly into two halves — a React frontend and an Express backend — that communicate through a REST API. The frontend never touches the database directly; it always goes through the API. This separation makes the codebase easier to maintain and means you could swap out the frontend or backend independently if needed.

4.1 How the Layers Fit Together

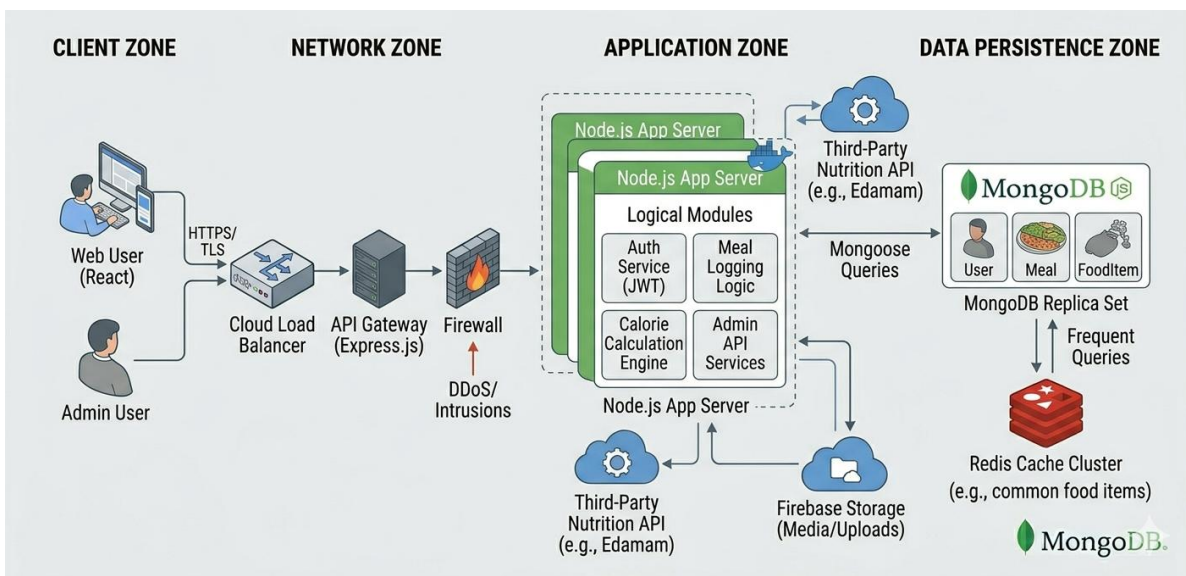
Layer	What It Does
Browser (React App)	What the user sees and interacts with — sends HTTP requests to the API using Axios

Express API Server	Runs on port 5000 — receives requests, checks auth, calls the right logic, returns JSON
Auth Routes	/api/auth — handles registration, login, and profile updates
Meal Routes	/api/meals — handles everything related to logging and managing meals
Food Routes	/api/foods — handles the food catalogue and search
MongoDB Database	Stores all users, foods, and meal logs persistently
JWT Middleware	Checks every protected request for a valid token before it reaches any logic

4.2 Technical Architecture Diagram

The diagram below shows how all the services connect — from the web app layer at the top, down through the API gateway and individual backend services, to the databases at the bottom.

Figure 1 — Technical Architecture Diagram



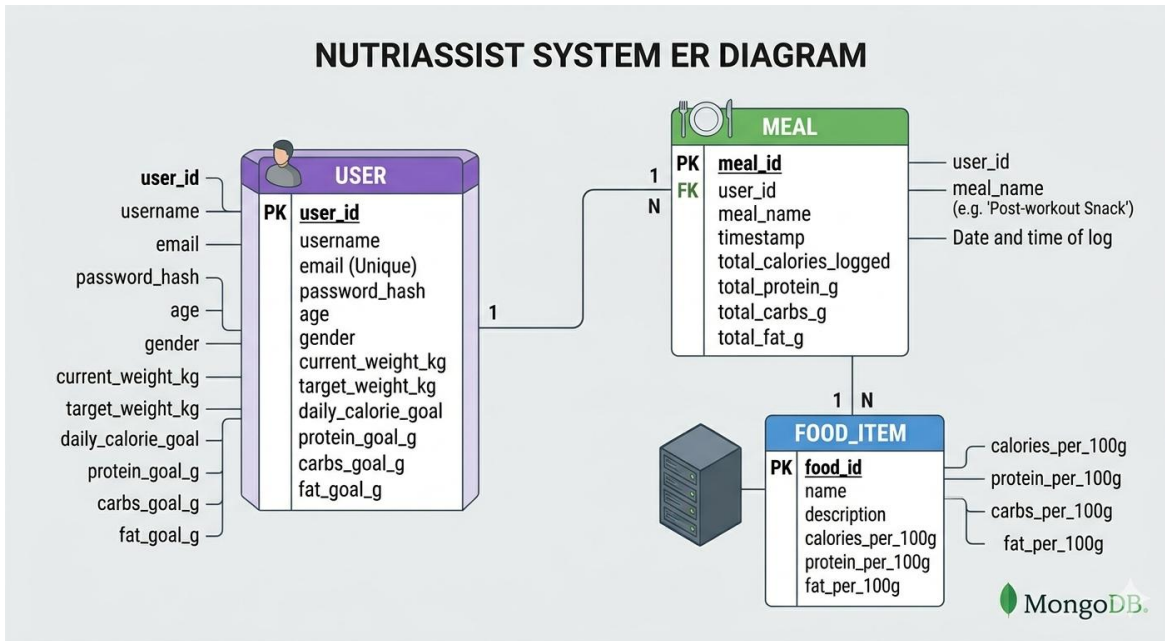
5. Database Design

We used MongoDB as the database because its flexible document model fits well with the kind of data this app handles — user profiles, food items, and meal logs each have slightly different shapes and evolve over time. Mongoose sits on top of MongoDB and lets us define clear schemas so the data stays consistent.

5.1 ER Diagram

The ER diagram below maps out all the entities in the system, their attributes, and how they relate to each other.

Figure 2 — ER Diagram



5.2 The Three Collections

The database has three main collections. Here is what each one holds and why it is structured the way it is.

5.2.1 Users

This collection stores everything about a registered user — their credentials, personal details, and their current health goal. The `targetCalories` field is computed automatically whenever the goal is set or changed, so users never have to calculate it themselves.

Field	What It Stores
<code>name</code>	Their full name
<code>email</code>	Unique — used to log in
<code>password</code>	Hashed with bcrypt before saving — never stored as plain text
<code>age / weight</code>	Optional fields used for profile display
<code>goal</code>	One of 8 options: <code>weight_loss</code> , <code>muscle_gain</code> , <code>maintain</code> , etc.
<code>targetCalories</code>	Daily calorie goal, auto-calculated from the chosen goal
<code>isAdmin</code>	Set to true for admin accounts — unlocks food management

5.2.2 Foods

Every food item — whether it was added by an admin or created privately by a user — lives in this collection. The `isGlobal` flag is what separates admin-published foods (visible to everyone) from a user's personal custom items.

Field	What It Stores
<code>name</code>	Food name — checked for duplicates (case-insensitive) on save
<code>unit_name</code>	Describes the serving size, e.g. '1 plate (300g)' or '1 piece'
<code>calories</code>	Kilocalories per unit
<code>protein / carbs / fats</code>	Grams per unit for each macronutrient
<code>isGlobal</code>	true = everyone can see it; false = only the creator can
<code>createdBy</code>	References the User who added it — null for pre-seeded global foods

5.2.3 Meals

Each document in this collection represents a single meal log entry. The macro values are computed and stored at the time of logging — multiplying the food's base values by the quantity entered — so the dashboard can query totals quickly without recalculating every time.

Field	What It Stores
<code>user</code>	References the User who logged this meal
<code>foodItemId</code>	References the Food that was logged
<code>name</code>	Food name stored directly — avoids an extra lookup on every dashboard load
<code>quantity</code>	How many units the user ate — macros are multiplied by this
<code>mealType</code>	Breakfast, Lunch, Dinner, or Snack
<code>calories / protein / carbs / fats</code>	Pre-computed values: food macro × quantity
<code>date</code>	When it was logged — defaults to now, used to filter meals by day

5.3 Pre-Loaded Food Data

When the server starts up, it checks whether the food collection has at least 10 global entries. If not, it wipes and re-seeds the collection with 13 common Indian foods so new users have something to work with right away, without needing to add everything manually.

- Biryani (Chicken) — 450 kcal per 300 g serving
- Masala Dosa — 415 kcal per dosa
- Curd Rice — 250 kcal per 200 g bowl
- Chapati — 104 kcal per piece
- Paneer Butter Masala — 350 kcal per 150 g
- Idli — 60 kcal per piece

- Sambar — 130 kcal per 150 g bowl
- Upma — 210 kcal per 150 g bowl
- Dal Tadka — 180 kcal per 150 g bowl
- Chicken Curry — 280 kcal per 200 g bowl
- Chicken Breast (Grilled) — 165 kcal per 100 g
- White Rice (Cooked) — 130 kcal per 100 g
- Egg (Whole) — 78 kcal per large egg

6. Roles, Responsibilities and User Flow

The app has two types of users — regular users who track their own meals, and admins who manage the food catalogue for everyone. Their permissions are separated at the API level so there is no way for a regular user to accidentally (or intentionally) modify the global food database.

6.1 What a Regular User Can Do

- Create an account and set up their profile with age, weight, and health goal.
- Browse and search the food catalogue, then log meals throughout the day.
- View their daily nutrition summary and weekly progress on the dashboard.
- Update their profile whenever their goal or stats change.
- Add their own private food items that only they can see and use.

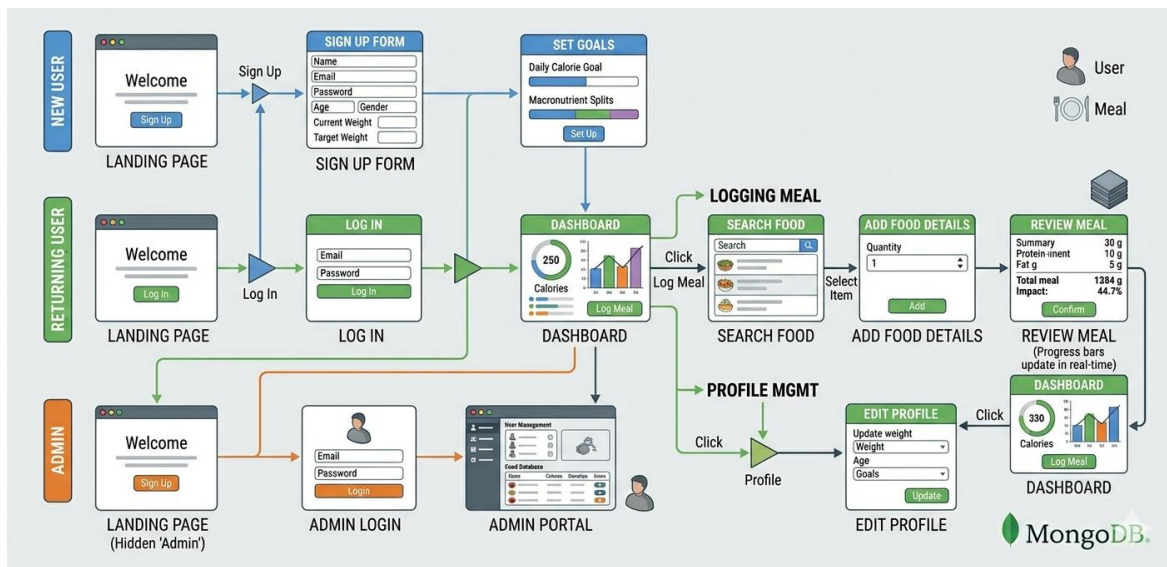
6.2 What an Admin Can Do

- Add new food items to the global catalogue that all users can access.
- Edit existing food entries to fix incorrect nutritional data.
- Delete outdated or duplicate food items from the database.
- Everything a regular user can do — admin accounts are not locked out of normal features.

6.3 User Flow Diagram

The diagram below traces the path a user takes through the app — from first landing on the site, through registration and daily use, to logging out. The admin flow is shown separately underneath.

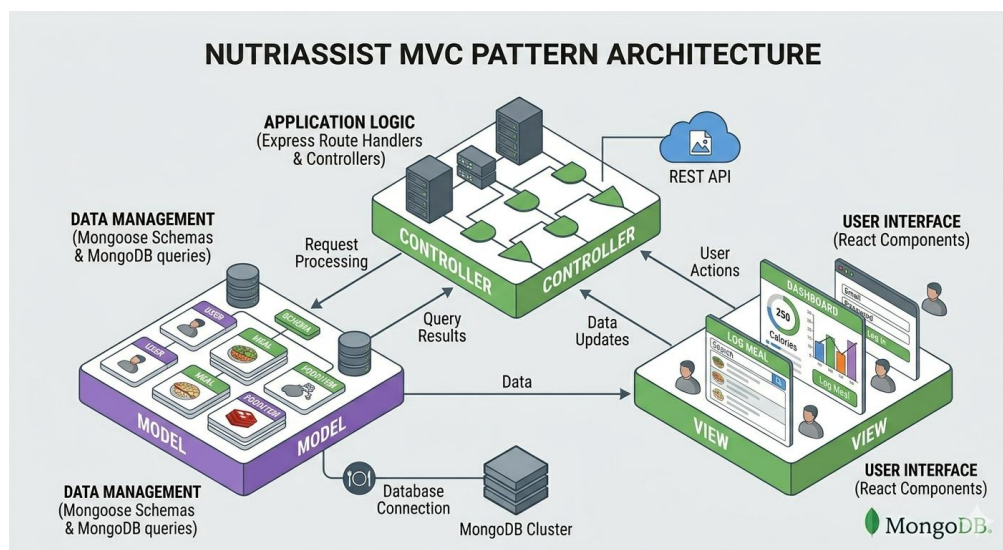
Figure 3 — User Flow Diagram



7. MVC Architecture Pattern

The backend is organised around the Model-View-Controller (MVC) pattern. If you have not worked with MVC before, the basic idea is simple: keep your data logic, your business logic, and your routing in separate files so they do not get tangled together. It makes the code much easier to read, test, and extend.

Figure 4 — MVC Pattern Diagram



DATABASE ↔ MODEL ↔ CONTROLLER ↔ VIEW (Routes) — USER sends Requests and receives Responses via Controller

7.1 Model — The Data Layer

Models are where the database schemas live. Each Mongoose model defines what a document looks like, what fields are required, and what validations apply. The models handle all direct communication with MongoDB — the rest of the code never touches the database directly.

File	What It Models
User.js	Defines the structure of a user account — credentials, goal, target calories, admin flag
Food.js	Defines a food item — name, serving unit, all four macro values, global/private flag
Meal.js	Defines a logged meal — links a user to a food, stores quantity and pre-computed macros

7.2 Controller — The Business Logic

Controllers contain the actual logic for each API action. When a request comes in, the router hands it to the right controller function, which validates the data, talks to the model, and sends back a response.

File	Functions It Contains
authController.js	registerUser, loginUser, updateProfile — handles the full user lifecycle including JWT creation
foodController.js	getFoods, createFood, updateFood, deleteFood, seedFoodsDB — manages the food catalogue
mealController.js	getMeals, setMeal, updateMeal, deleteMeal — logs meals and computes macros on the fly

7.3 View — The Routing Layer

In a REST API, the 'View' is the routing layer. Express routers define which URL maps to which controller function, and which routes need authentication before they can be accessed.

File	Routes It Defines
authRoutes.js	POST /register, POST /login, PUT /profile
foodRoutes.js	GET /, POST /, PUT /:id, DELETE /:id (mounted at /api/foods)
mealRoutes.js	GET /, POST /, PUT /:id, DELETE /:id (mounted at /api/meals)

7.4 Why MVC Made Sense for This Project

- **Clean separation:** Each file has one job. You can read authController.js and know exactly what it does without looking at anything else.

- **Easy to extend:** Adding a new feature means adding a new route, a new controller function, and possibly a new model — existing code stays untouched.
- **Reusable logic:** The same controller functions handle both admin and user requests — permissions are checked by middleware, not duplicated in every function.
- **Straightforward testing:** You can test a controller function in isolation without spinning up the whole server.
- **Team-friendly:** Multiple people can work on models, controllers, and routes simultaneously without constantly stepping on each other's changes.

8. Overall Project Architecture

This section gives a bird's-eye view of the full project — how the frontend folders connect to the backend folders, and how everything ultimately talks to MongoDB.

Project Architecture

8.1 Backend Folder Structure

```
Backend/
├── config/
│   └── db.js           # Connects to MongoDB on startup
├── controllers/
│   ├── authController.js # Register, login, update profile
│   ├── foodController.js # Food catalogue CRUD + seeding
│   └── mealController.js  # Meal logging and macro computation
├── middleware/
│   ├── authMiddleware.js # JWT verification + admin check
│   ├── errorMiddleware.js # Catches and formats all errors
│   └── validationMiddleware.js # Runs express-validator checks
├── models/
│   ├── User.js / Food.js / Meal.js
├── routes/
│   ├── authRoutes.js / foodRoutes.js / mealRoutes.js
├── .env                # Secrets: MONGO_URI, JWT_SECRET, PORT
└── server.js           # Entry point — boots the Express app
```

8.2 Frontend Folder Structure

```
Frontend/src/
├── components/
│   ├── Navbar.jsx      # Top nav — shows different links based on
login state
│   └── ProtectedRoute.jsx # Redirects to /login if user is not
authenticated
├── pages/
│   ├── Home.jsx / Register.jsx / Login.jsx
```

```

|   |   | Dashboard.jsx / MealTracker.jsx
|   |   | Profile.jsx / AdminFoods.jsx
|   |   |
|   |   | └─ routes/
|   |   |   |   └─ AppRoutes.jsx          # Defines all routes, wraps private ones in
ProtectedRoute
|   |   |   |   └─ services/
|   |   |   |       |   └─ AuthContext.jsx      # Stores logged-in user globally, provides
login/logout
|   |   |   |       |   └─ api.js              # Axios instance — adds the JWT header to
every request
|   |   |   |       └─ App.jsx                # Root component — wraps everything in
AuthProvider

```

9. Backend Development

The backend is a Node.js/Express REST API. Every route that involves user data is protected by JWT middleware, so unauthenticated requests are rejected before they reach any business logic. Below is a breakdown of each controller and the API routes.

9.1 Authentication

Three endpoints handle everything related to user accounts.

Function	What It Does
registerUser	Checks if the email is already taken, hashes the password with bcrypt, computes the target calories from the chosen goal, saves the user, and returns a JWT.
loginUser	Finds the user by email, compares the submitted password against the stored hash using bcrypt.compare, and returns a JWT along with the user's profile data.
updateProfile	Protected endpoint — updates name, age, weight, or goal, and recalculates targetCalories if the goal changed.

9.2 Food Catalogue

These endpoints manage the food database — both the global catalogue and private user foods.

Function	What It Does
getFoods	Returns all global foods plus the requesting user's private foods. Supports a ?search= query parameter that does a case-insensitive name match.
createFood	Admins create global foods; regular users create private ones. Rejects duplicates by checking for an existing food with the same name.
updateFood	Admins can edit any food. Regular users can only edit their own private foods.

deleteFood	Same ownership rules as update — you can only delete what you own.
seedFoodsDB	Runs automatically when the server starts. If fewer than 10 global foods exist, it clears and re-seeds the collection with 13 Indian staples.

9.3 Meal Logging

These endpoints handle everything related to logging and managing a user's meals.

Function	What It Does
getMeals	Returns all meals for the logged-in user, sorted newest first.
setMeal	Looks up the food by ID, multiplies each macro value by the quantity entered, then saves the resulting Meal document.
updateMeal	Verifies the meal belongs to the requesting user, then updates the fields — recalculating macros if the food or quantity changed.
deleteMeal	Verifies ownership before deleting. A user cannot delete another user's meal.

9.4 Middleware

Middleware	What It Does
protect	Reads the Bearer token from the Authorization header, verifies it with <code>jwt.verify</code> , and attaches the decoded user to <code>req.user</code> . Rejects the request if the token is missing or invalid.
admin	Runs after <code>protect</code> . Checks whether <code>req.user.isAdmin</code> is true — returns a 401 error if not.
validate	Wraps <code>express-validator</code> 's <code>validationResult</code> . If any validation rule failed, it returns a 400 response with the list of errors before the controller even runs.
errorHandler	A catch-all Express error handler. Any error passed to <code>next()</code> ends up here and gets returned as a clean JSON response with an appropriate status code.

9.5 API Endpoints at a Glance

Endpoint	What It Does
POST /api/auth/register	Create a new account — no auth needed
POST /api/auth/login	Log in and receive a JWT — no auth needed
PUT /api/auth/profile	Update your profile — must be logged in
GET /api/foods	Browse and search foods — must be logged in
POST /api/foods	Add a new food — must be logged in

PUT /api/foods/:id	Edit a food — must own it or be admin
DELETE /api/foods/:id	Remove a food — must own it or be admin
GET /api/meals	View your meal history — must be logged in
POST /api/meals	Log a meal — must be logged in
PUT /api/meals/:id	Edit a logged meal — must own it
DELETE /api/meals/:id	Delete a logged meal — must own it

10. Frontend Development

The frontend is a single-page React application styled with Tailwind CSS. It communicates with the backend through a centralised Axios instance that automatically attaches the user's JWT to every request — so individual components never have to worry about authentication headers.

10.1 Libraries and Tools Used

Library	Why We Used It
React.js v18	Component-based UI — the whole frontend is built with this
Vite	Starts up in under a second and refreshes the browser instantly on save
Tailwind CSS	Utility classes for styling — no separate CSS files needed
React Router	Handles navigation between pages without full page reloads
Axios	Sends HTTP requests to the API; the instance in api.js injects the JWT header automatically
Recharts	Powers the PieChart and BarChart on the dashboard
React Icons	Provides all the small icons used throughout the UI
React Toastify	Shows non-intrusive success and error notifications

10.2 Page-by-Page Breakdown

- **Home.jsx:** The public landing page. Introduces the app with a short pitch and two call-to-action buttons — Sign Up and Log In. Anyone can see this page without being logged in.
- **Register.jsx:** A simple registration form that collects name, email, password, and optionally age and weight. The goal dropdown has 8 options, and the app sets the calorie target as soon as the user submits.
- **Login.jsx:** Email and password form. On a successful login the JWT is stored in AuthContext and the user is redirected straight to their dashboard.

- **Dashboard.jsx:** The heart of the app. Shows a personalised greeting with the user's goal and daily calorie target. Below that is the Smart Assistant Hub with three panels: today's macro analysis, personalised food suggestions, and a weekly habit consistency score. The page also has a calorie progress bar, individual macro progress bars, a donut PieChart showing today's macro split, and a 7-day BarChart of calorie history.
- **MealTracker.jsx:** Where users actually log their meals. Type in a food name to search the catalogue, pick a quantity and meal type, and hit add. The meal appears in a list below with edit and delete options.
- **Profile.jsx:** A pre-filled form showing the user's current details. Changing the goal here automatically recalculates the daily calorie target on the server side.
- **AdminFoods.jsx:** Only visible to admin accounts. Displays the full global food catalogue in a table, with an Add Food form at the top and edit/delete buttons on each row.

11. Key Features

Here is a summary of the main things the app does that make it worth using.

6. **Goal-based calorie targets** — Pick a goal and the app figures out your daily calorie target for you. Change your goal later and it updates automatically.
7. **Real-time macro tracking** — Log a meal and the dashboard updates instantly — progress bars for calories, protein, carbs, and fats all refresh on the spot.
8. **Smart Assistant Hub** — Three panels that give you actionable information: what your intake looks like today, what foods would help based on your goal, and how consistent you have been this week.
9. **Context-aware daily insight** — A message on the dashboard that changes throughout the day based on your goal, how much you have eaten, and where your macros stand.
10. **Nutrition charts** — A donut chart showing today's macro split, and a bar chart showing the last seven days of calorie data. Bars go red on days you exceeded your target.
11. **Indian food catalogue** — 13 common Indian foods are pre-loaded so you have something useful from day one — no setup required.
12. **Custom food items** — Add your own foods if they are not in the catalogue. Your private items stay private unless you are an admin publishing them globally.
13. **Admin food management** — Admin accounts get a dedicated page to manage everything in the global catalogue — add, edit, and delete, all in one place.
14. **JWT security** — Tokens last 30 days. All passwords are hashed with bcrypt. Every protected route checks the token before doing anything.
15. **Input validation everywhere** — The API validates incoming data before it reaches the database, so bad data never gets stored.

12. Project Setup and Configuration

Getting the project running locally takes about five minutes. You need Node.js and MongoDB installed first — everything else installs automatically via npm.

12.1 Setting Up the Backend

16. Open a terminal and navigate to the backend folder: `cd backend`
17. Install all dependencies: `npm install`
18. Create a file called `.env` in the backend folder and add these three lines:
 `MONGO_URI=<your MongoDB connection string>`
 `JWT_SECRET=<any long random string>`
 `PORT=5000`
19. Start the server: `npm run dev` — you should see 'Server running on port 5000' and 'MongoDB connected'

12.2 Setting Up the Frontend

20. Open a new terminal and navigate to the frontend folder: `cd frontend`
21. Install all dependencies: `npm install`
22. Open `src/services/api.js` and confirm the `baseUrl` is set to `http://localhost:5000/api`
23. Start the dev server: `npm run dev` — the app opens at `http://localhost:5173`

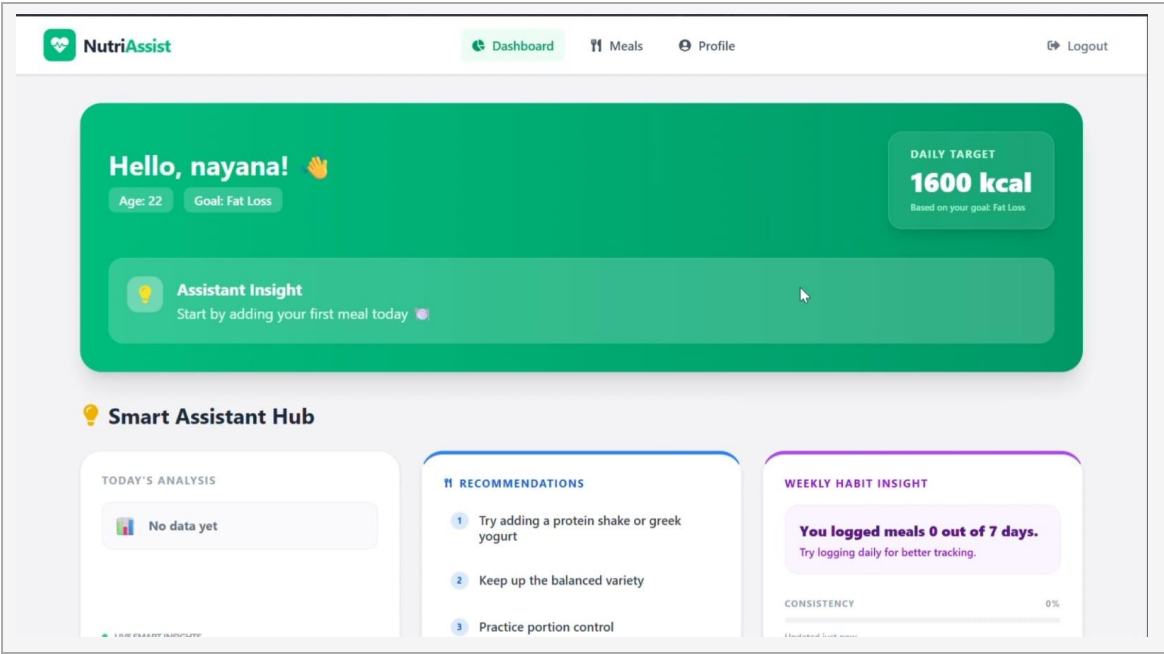
12.3 Testing the API with Postman

The `postman` folder contains a ready-made collection covering every endpoint, plus an environment file with the base URL pre-configured. Import both files into Postman, set the `{{baseUrl}}` variable to `http://localhost:5000/api`, and run the requests in order — register first to get a token, then use that token for the protected routes.

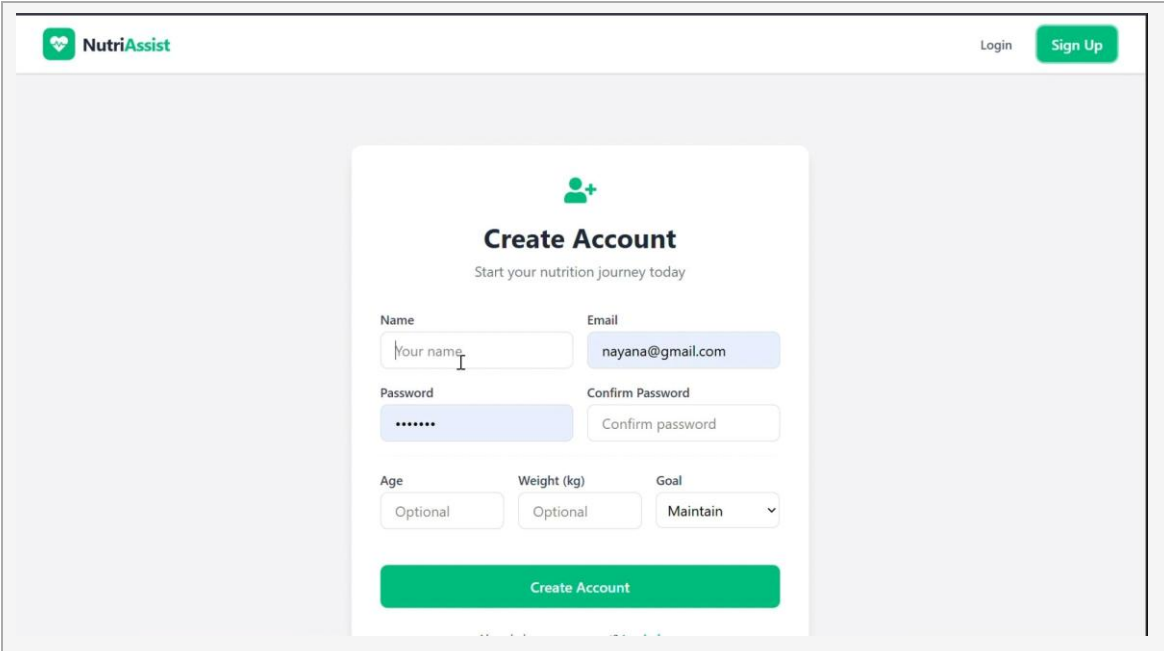
13. Output Screenshots

The placeholders below mark where each screen of the application should appear. Click into any box and paste or insert your screenshot directly.

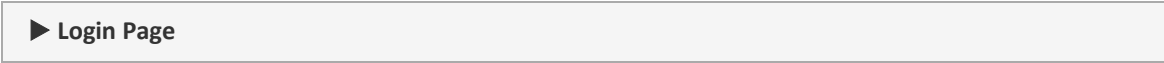
Home Page



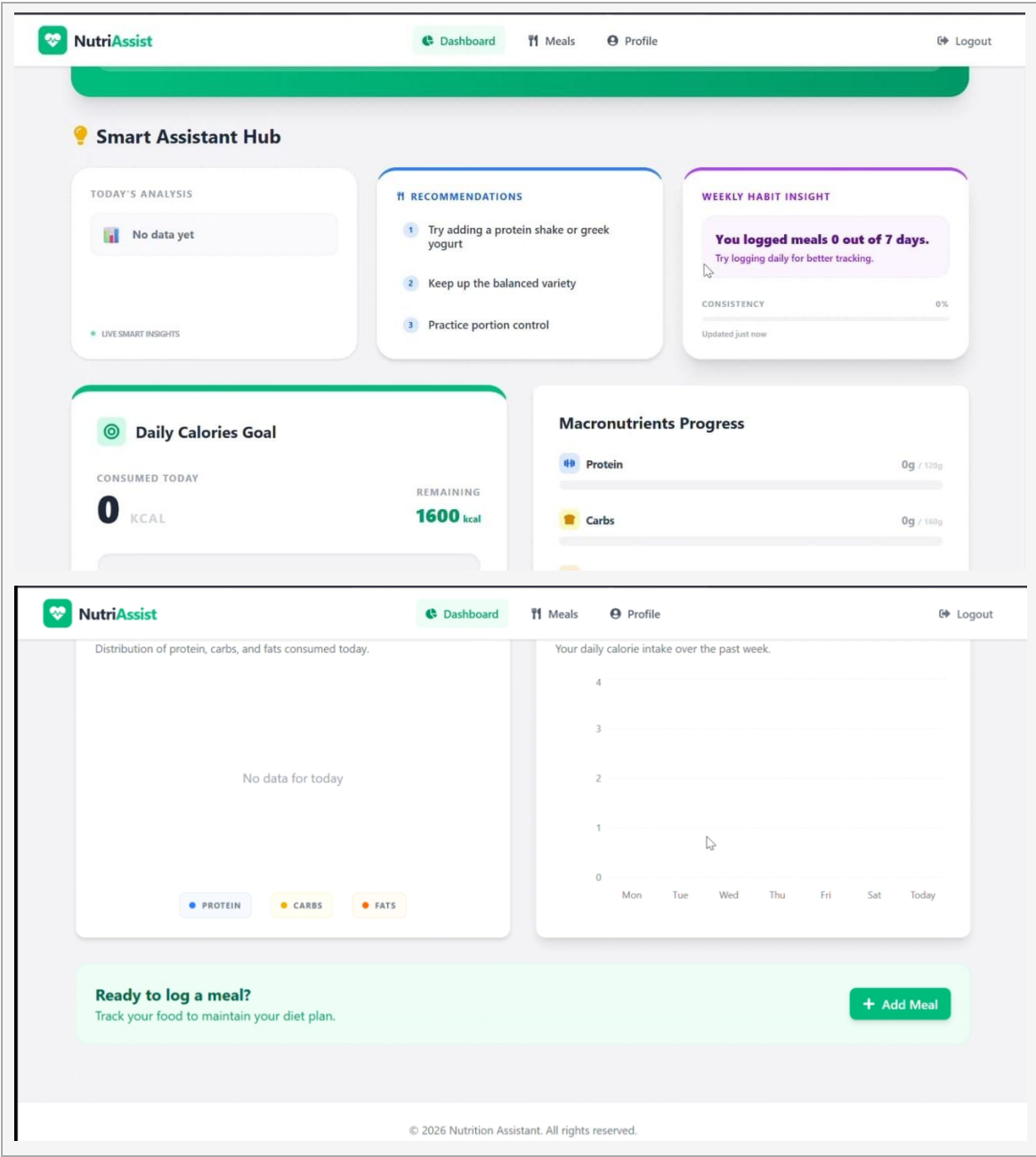
Register Page



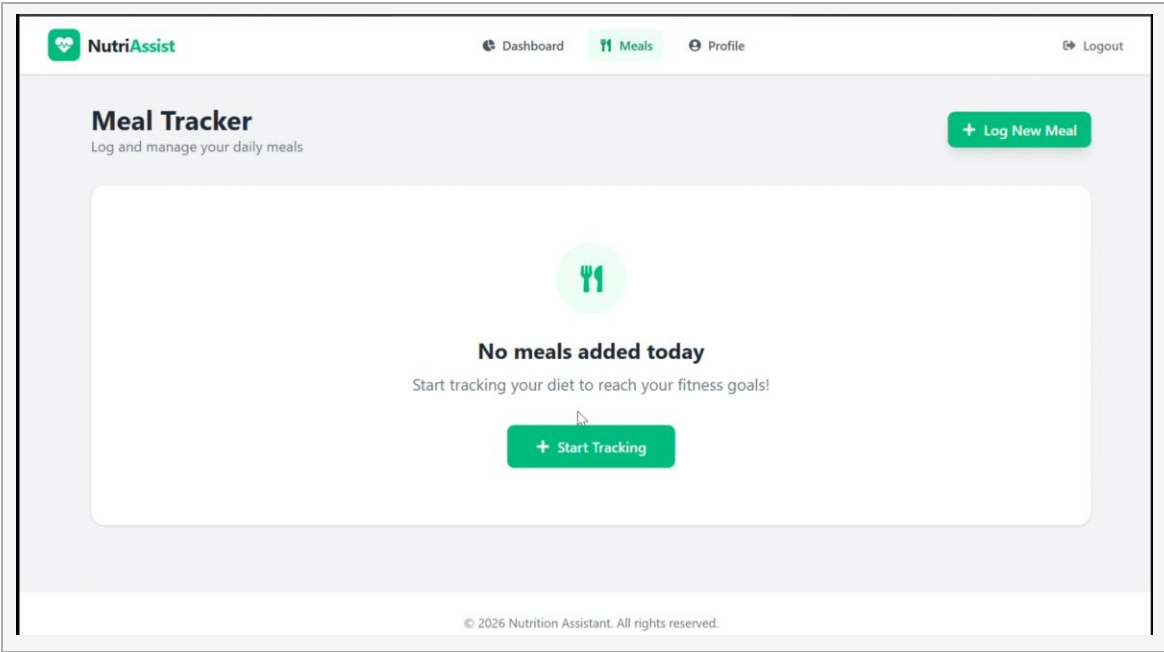
Login Page



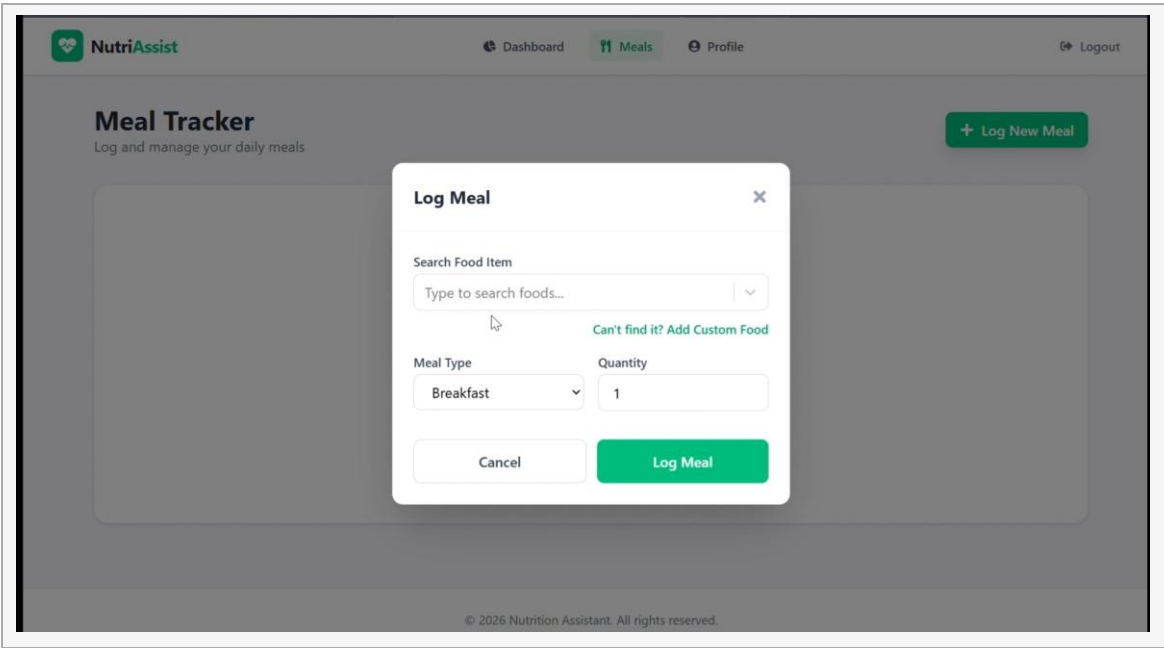
Dashboard



Meal Tracker Page



Logging Meal Page



Add Item Page

NutriAssist

Dashboard

Meals

Profile

Logout

Meal Tracker

Log and manage your daily meals

+ Log New Meal

Log Meal

Search Food Item

Egg (Whole) (1 large) - 78 kcal

Can't find it? Add Custom Food

Meal Type

Breakfast

Quantity

1

ESTIMATED NUTRITION

Cal: 78

Pro: 6g

Carbs: 0.6g

Fat: 5g

Cancel

Log Meal

© 2026 Nutrition Assistant. All rights reserved.

After adding meal

NutriAssist

Dashboard

Meals

Profile

Logout

Meal Tracker

Log and manage your daily meals

+ Log New Meal

Breakfast

Egg (Whole)

78 kcal

PROTEIN

6g

CARBS

0.6g

FATS

5g

Quantity: 1

19/04/2026

Edit

Delete

Meal logged successfully

© 2026 Nutrition Assistant. All rights reserved.

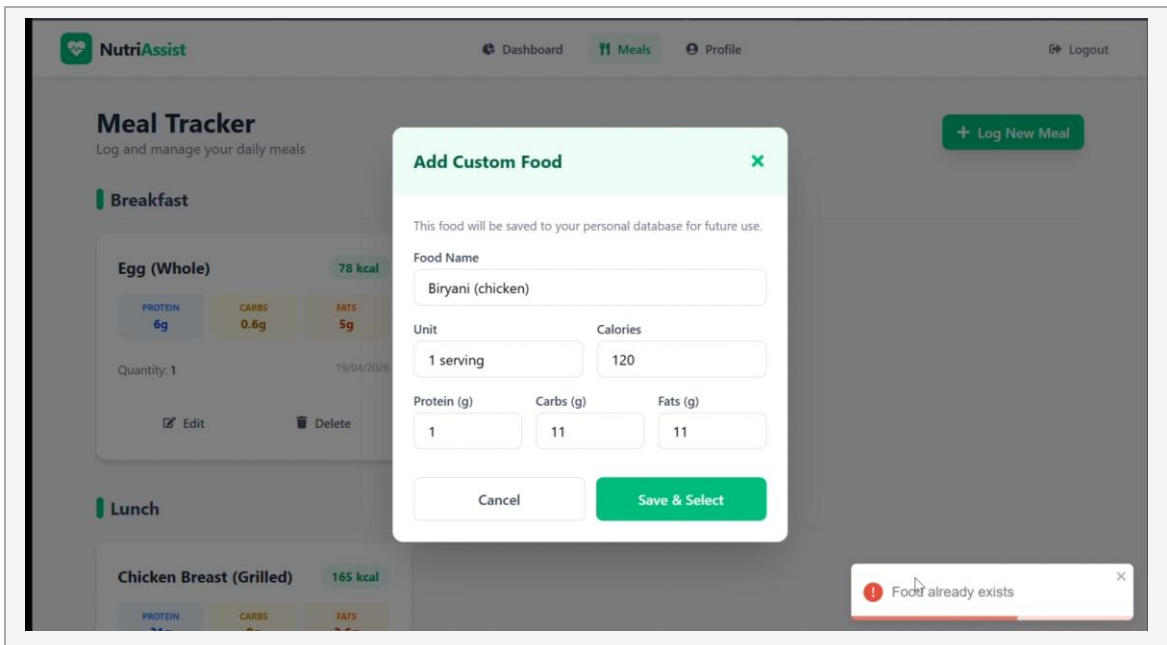
Customizing Meal Option

The screenshot shows the NutriAssist web application. The top navigation bar includes 'Dashboard', 'Meals', 'Profile', and 'Logout'. The main heading is 'Meal Tracker' with the subtitle 'Log and manage your daily meals'. A '+ Log New Meal' button is in the top right. The 'Breakfast' section is active, displaying 'Egg (Whole)' with 78 kcal, 6g protein, 0.6g carbs, and 5g fats. Below this, it shows 'Quantity: 1' and a date '19/04/2026'. A 'Lunch' section is partially visible below, showing 'Chicken Breast (Grilled)' with 165 kcal. A modal titled 'Add Custom Food' is open in the center. It contains a message: 'This food will be saved to your personal database for future use.' The form fields are: 'Food Name' (with placeholder 'e.g. Mom's Special Curry'), 'Unit' (set to '1 serving'), 'Calories' (empty), 'Protein (g)' (empty), 'Carbs (g)' (empty), and 'Fats (g)' (empty). At the bottom of the modal are 'Cancel' and 'Save & Select' buttons.

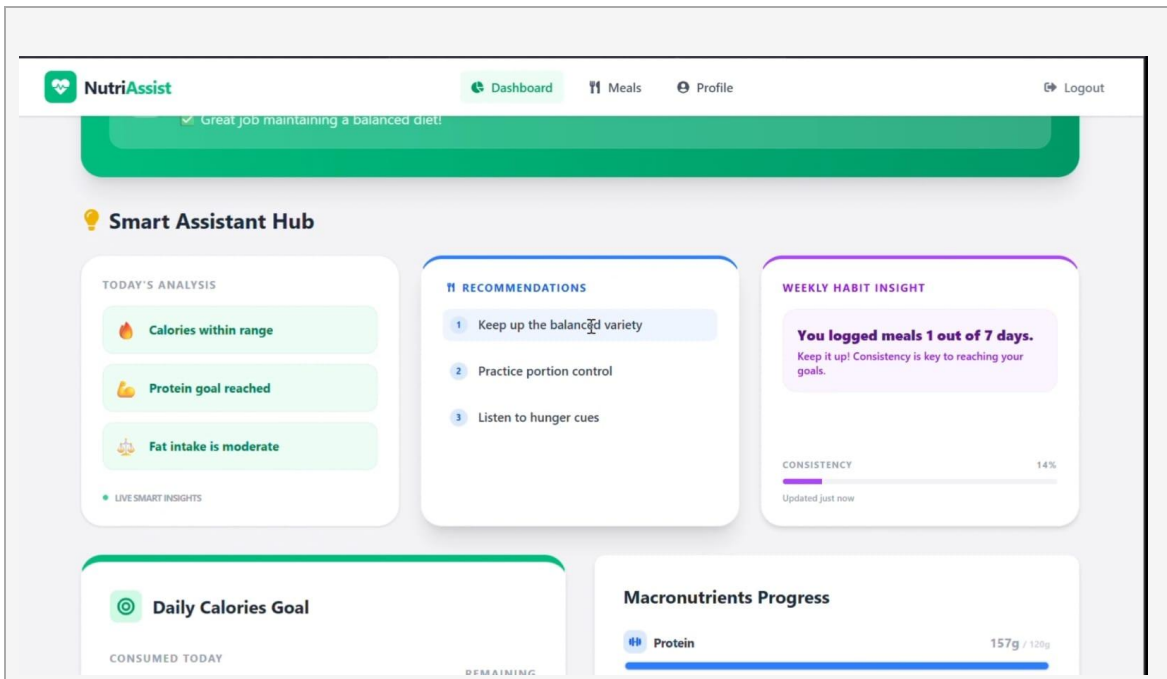
After Customization

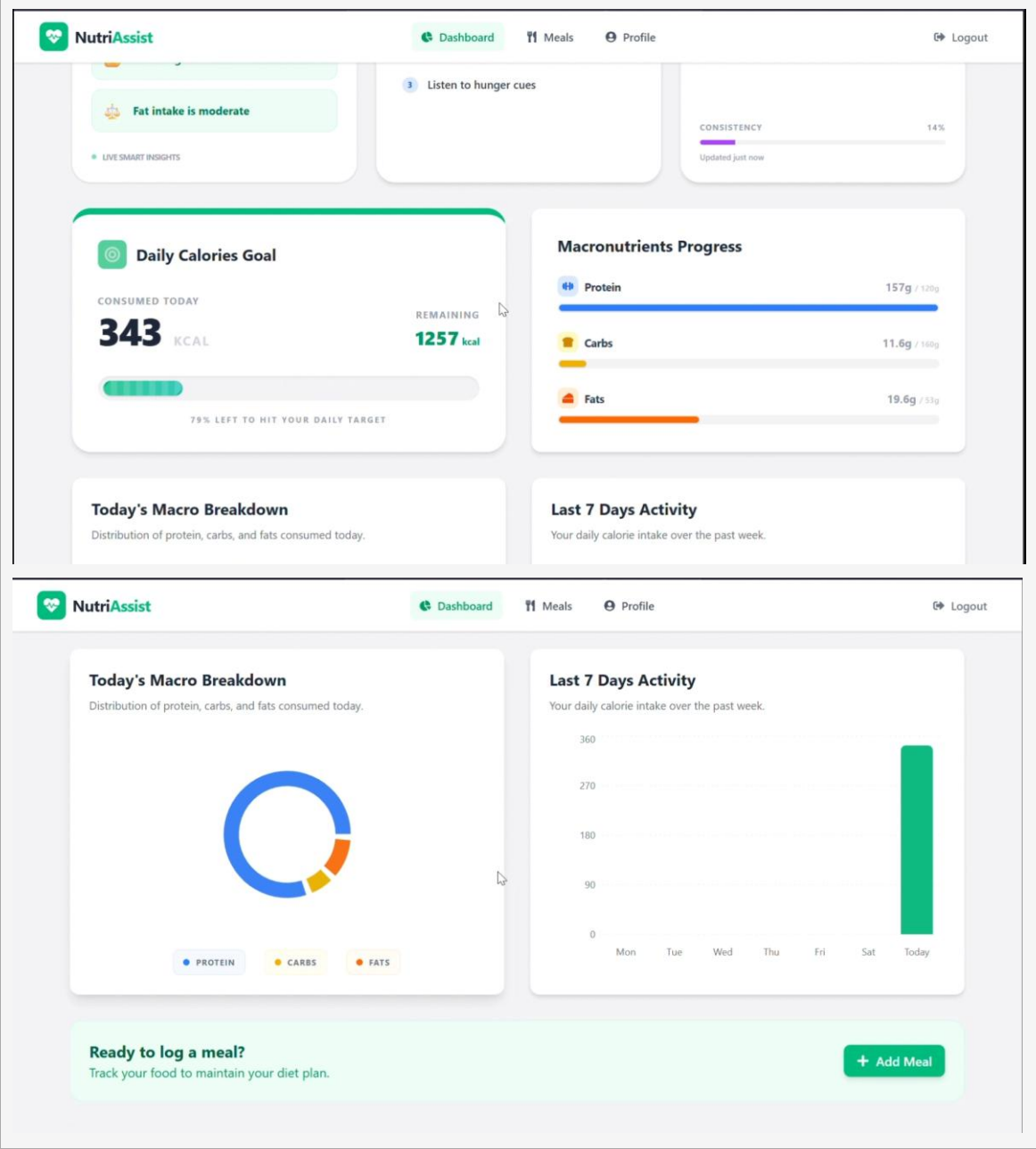
This screenshot shows the same NutriAssist interface after the food item has been customized. The 'Add Custom Food' modal is still open. The 'Food Name' field now contains 'mangoes'. The 'Unit' is still '1 serving', but the 'Calories' field now contains '100'. The 'Protein (g)' field contains '120', 'Carbs (g)' contains '11', and the 'Fats (g)' field contains '11' with a spinner control. The 'Save & Select' button is highlighted with a mouse cursor. The background meal tracker interface remains the same as in the previous screenshot.

Response to repeated food addition

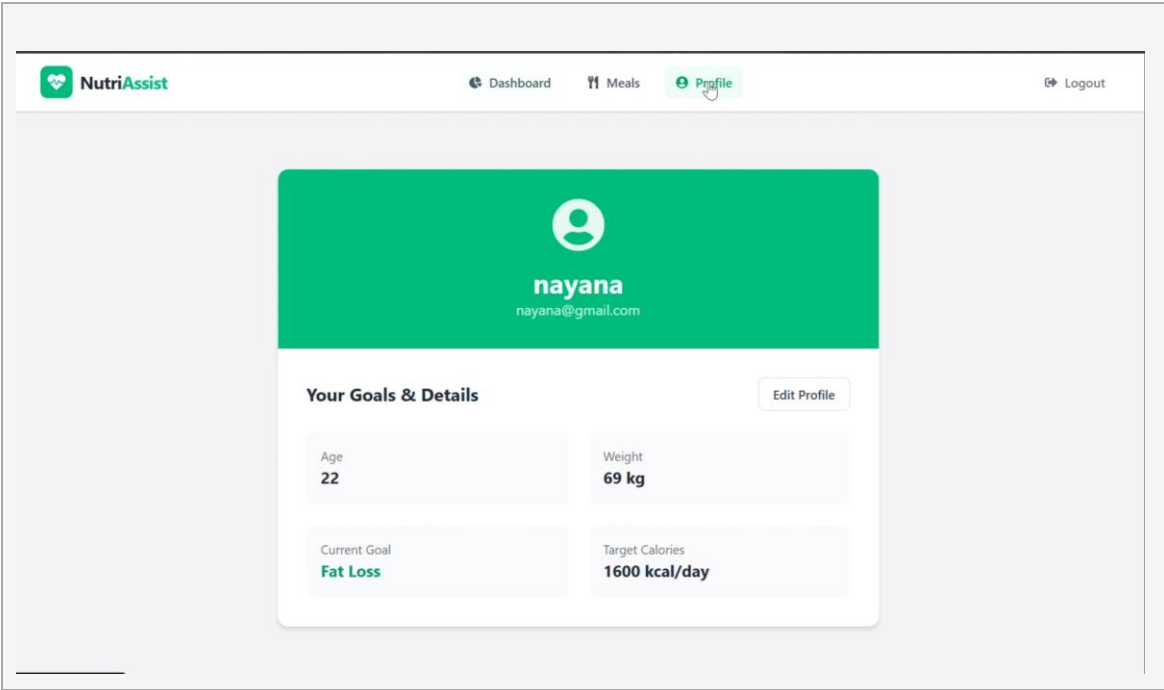


After logging Meals/ Dashboard

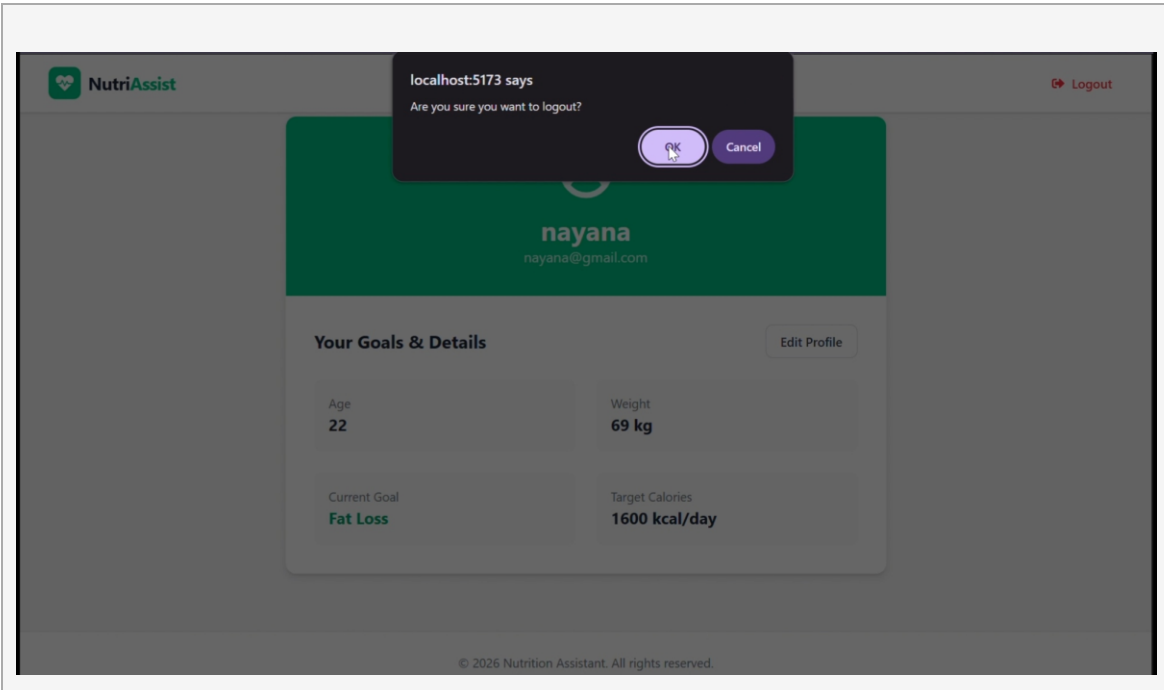




Profile Page



Logout Page



14. Code Snippets

The placeholders below mark where each important code file should be shown. Paste a screenshot of the code or type the listing directly into each box.

server.js — Express App Entry Point

(Express setup, CORS config, middleware, route mounting, port listen)

```
</> server.js — Express App Entry Point

dotenv.config();

connectDB().then(() => {
  |   seedFoodsDB();
});

const app = express();

app.use(cors());
app.use(express.json());

app.use('/api/auth', require('./routes/authRoutes'));
app.use('/api/meals', require('./routes/mealRoutes'));
app.use('/api/foods', require('./routes/foodRoutes'));

app.use(errorHandler);

const PORT = process.env.PORT || 5000;

app.listen(PORT, () => {
  |   console.log(`Server running on port ${PORT}`);
});
```

config/db.js — Database Connection

(Async connectDB() with Mongoose — logs success or exits on failure)

</> config/db.js — Database Connection

```
const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI);
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`Error: ${error.message}`);
    process.exit(1);
  }
};
```

models/User.js — User Schema

(Schema with goal enum, targetCalories, isAdmin, and bcrypt pre-save hook)

</> models/User.js — User Schema

```

const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Please add a name']
  },
  email: {
    type: String,
    required: [true, 'Please add an email'],
    unique: true
  },
  password: {
    type: String,
    required: [true, 'Please add a password']
  },
  age: {
    type: Number,
    default: null
  },
  weight: {
    type: Number,
    default: null
  },
  goal: {
    type: String,
    enum: [
      'weight_loss',
      'fat_loss',
      'muscle_gain',
      'weight_gain',
      'maintain',
      'improve_fitness',
      'balanced_diet',
      'high_protein_diet'
    ],
    default: 'maintain'
  },
  targetCalories: {
    type: Number,
    default: 2000
  },
  isAdmin: {
    type: Boolean,
    default: false
  }
}, {
  timestamps: true
});

// Hash password before saving
userSchema.pre('save', async function (next) {
  if (!this.isModified('password')) {
    next();
  }
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
});

```

models/Food.js — Food Schema

(Schema with *isGlobal* flag and *createdBy* ObjectId reference)

</> models/Food.js — Food Schema

```
const foodSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true,
    unique: true
  },
  unit_name: {
    type: String,
    required: true // e.g., '100g', '1 medi
  },
  calories: {
    type: Number,
    required: true
  },
  protein: {
    type: Number,
    required: true
  },
  carbs: {
    type: Number,
    required: true
  },
  fats: {
    type: Number,
    required: true
  },
  isGlobal: {
    type: Boolean,
    default: true
  },
  createdBy: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    default: null
  }
}, {
  timestamps: true
});
```

models/Meal.js — Meal Schema

(Schema with quantity multiplier and pre-computed macro fields)

</> models/Meal.js — Meal Schema

```
const mealSchema = new mongoose.Schema({
  user: {
    type: mongoose.Schema.Types.ObjectId,
    required: true,
    ref: 'User'
  },
  foodItemId: {
    type: mongoose.Schema.Types.ObjectId,
    (property) ref: string
    ref: 'Food'
  },
  quantity: {
    type: Number,
    required: [true, 'Please add a quantity'],
    default: 1
  },
  name: {
    type: String,
    required: true
  },
  mealType: {
    type: String,
    enum: ['BreakFast', 'Lunch', 'Dinner', 'Snack'],
    default: 'Snack'
  },
  calories: {
    type: Number,
    required: true
  },
  protein: {
    type: Number,
    required: true
  },
  carbs: {
    type: Number,
    required: true
  },
  fats: {
    type: Number,
    required: true
  },
  date: {
    type: Date,
    default: Date.now
  }
}, {
  timestamps: true
});
```

controllers/authController.js — registerUser

(Email uniqueness check, bcrypt hash, targetCalories from goal, JWT return)

</> controllers/authController.js — registerUser

```

const registerUser = async (req, res, next) => {
  try {
    const { name, email, password, age, weight, goal } = req.body;

    // Check if user exists
    const userExists = await User.findOne({ email });

    if (userExists) {
      res.status(400);
      throw new Error('User already exists');
    }

    const targetCalories = getTargetCalories(goal || 'maintain');

    // Create user
    const user = await User.create({
      name,
      email,
      password,
      age: age || null,
      weight: weight || null,
      goal: goal || 'maintain',
      targetCalories
    });

    if (user) {
      res.status(201).json({
        _id: user._id,
        name: user.name,
        email: user.email,
        age: user.age,
        weight: user.weight,
        goal: user.goal,
        targetCalories: user.targetCalories,
        isAdmin: user.isAdmin,
        token: generateToken(user._id),
      });
    } else {
      res.status(400);
      throw new Error('Invalid user data');
    }
  } catch (error) {
    next(error);
  }
};

```

controllers/authController.js — loginUser

(bcrypt.compare, JWT sign and return with user profile)

</> controllers/authController.js — loginUser

```

const loginUser = async (req, res, next) => {
  try {
    const { email, password } = req.body;

    // Check for user email
    const user = await User.findOne({ email });

    if (user && (await user.matchPassword(password))) {
      res.json({
        _id: user._id,
        name: user.name,
        email: user.email,
        age: user.age,
        weight: user.weight,
        goal: user.goal,
        targetCalories: user.targetCalories,
        isAdmin: user.isAdmin,
        token: generateToken(user._id),
      });
    } else {
      res.status(401);
      throw new Error('Invalid credentials');
    }
  } catch (error) {
    next(error);
  }
};

```

controllers/foodController.js — getFoods

(isGlobal OR createdBy filter merged with optional regex search)

```
</> controllers/foodController.js — getFoods
```

```
const getFoods = async (req, res, next) => {
  try {
    const keyword = req.query.search ? {
      name: {
        $regex: req.query.search,
        $options: 'i'
      }
    } : {};

    // Fetch global foods OR user-created foods
    const query = {
      ...keyword,
      $or: [
        { isGlobal: true },
        { createdBy: req.user.id }
      ]
    };

    const foods = await Food.find(query).sort({ name: 1 });
    res.json(foods);
  } catch (error) {
    next(error);
  }
};
```

controllers/mealController.js — setMeal

(Food lookup, $\text{macro} = \text{food.value} \times \text{quantity}$, Meal.create)

</> controllers/mealController.js — setMeal

```
const setMeal = async (req, res, next) => {
  try {
    const { foodItemId, quantity, mealType } = req.body;

    // Fetch food item to calculate macros
    const food = await Food.findById(foodItemId);
    if (!food) {
      res.status(404);
      throw new Error('Food item not found');
    }

    const calculatedCalories = Math.round(food.calories * quantity);
    const calculatedProtein = Math.round(food.protein * quantity * 10) / 10;
    const calculatedCarbs = Math.round(food.carbs * quantity * 10) / 10;
    const calculatedFats = Math.round(food.fats * quantity * 10) / 10;

    const meal = await Meal.create({
      user: req.user.id,
      foodItemId,
      name: food.name,
      quantity,
      mealType: mealType || 'Snack',
      calories: calculatedCalories,
      protein: calculatedProtein,
      carbs: calculatedCarbs,
      fats: calculatedFats
    });

    res.status(201).json(meal);
  } catch (error) {
    next(error);
  }
};
```


middleware/authMiddleware.js — protect

(Bearer token extraction, jwt.verify, req.user attachment)

</> middleware/authMiddleware.js — protect

```
const protect = async (req, res, next) => {
  let token;

  if (
    req.headers.authorization &&
    req.headers.authorization.startsWith('Bearer')
  ) {
    try {
      // Get token from header
      token = req.headers.authorization.split(' ')[1];

      // Verify token
      const decoded = jwt.verify(token, process.env.JWT_SECRET);

      // Get user from the token
      req.user = await User.findById(decoded.id).select('-password');

      next();
    }
  }
}
```

frontend/AuthContext.jsx

(React Context: user state, login(), logout(), localStorage persistence)

</> frontend/AuthContext.jsx

```
export const AuthContext = createContext();

export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const storedUser = localStorage.getItem('user');
    if (storedUser) {
      setUser(JSON.parse(storedUser));
    }
    setLoading(false);
  }, []);

  const login = async (email, password) => {
    const response = await api.post('auth/login', { email, password });
    if (response.data) {
      localStorage.setItem('user', JSON.stringify(response.data));
      setUser(response.data);
    }
    return response.data;
  };

  const register = async (name, email, password, age, weight, goal) => {
    const response = await api.post('auth/register', { name, email, password, age, weight, goal });
    if (response.data) {
      localStorage.setItem('user', JSON.stringify(response.data));
      setUser(response.data);
    }
    return response.data;
  };
};
```

frontend/Dashboard.jsx — Smart Assistant Hub

(getDailyAnalysis(), getPersonalizedSuggestions(), getWeeklyHabits() logic)

```
</> frontend/Dashboard.jsx — Smart Assistant Hub
```

```
const totalNutrition = todaysMeals.reduce(
  (acc, meal) => ({
    calories: acc.calories + meal.calories,
    protein: acc.protein + meal.protein,
    carbs: acc.carbs + meal.carbs,
    fats: acc.fats + meal.fats,
  }),
  { calories: 0, protein: 0, carbs: 0, fats: 0 }
);
```

frontend/Dashboard.jsx — Recharts Charts

(PieChart for macro split + BarChart for 7-day calorie history)

```
</> frontend/Dashboard.jsx — Recharts Charts
```

```
<BarChart data={getLast7DaysData()}>
  <CartesianGrid strokeDasharray="3 3" vertical={false} stroke="#f3f4f6" />
  <XAxis dataKey="label" axisLine={false} tickLine={false} tick={{fill: '#6b7280', fontSize: 12}} dy={10} />
  <YAxis axisLine={false} tickLine={false} tick={{fill: '#6b7280', fontSize: 12}} />
  <RechartsTooltip
    cursor={{fill: '#f3f4f6'}}
    contentStyle={{borderRadius: '8px', border: 'none', boxShadow: '0 4px 6px -1px rgb(0 0 0 / 0.1)}}
  />
  <Bar
    dataKey="calories"
    fill="#10b981"
    radius={[6, 6, 0, 0]}
    maxBarSize={40}
    name="Calories"
  >
    {getLast7DaysData().map((entry, index) => (
      <Cell
        key={`cell-${index}`}
        fill={entry.calories > targetCalories ? '#ef4444' : '#10b981'}
        className="transition-all duration-500"
      />
    ))}
  </Bar>
</BarChart>
```

frontend/api.js — Axios Instance

(baseUrl config and request interceptor that injects the JWT header)

</> frontend/api.js — Axios Instance

```
const api = axios.create({
  |   baseUrl: API_URL,
  | });

// Add a request interceptor to attach JWT token
api.interceptors.request.use(
  |   (config) => {
  |     |   const userStr = localStorage.getItem('user');
  |     |   if (userStr) {
  |     |     |   const user = JSON.parse(userStr);
  |     |     |   if (user.token) {
  |     |     |     |   config.headers.Authorization = `Bearer ${user.token}`;
  |     |     |   }
  |     |   }
  |     |   return config;
  |   },
  |   (error) => {
  |     |   return Promise.reject(error);
  |   }
  | );
```

frontend/AppRoutes.jsx

(All route definitions with `ProtectedRoute` wrappers for private pages)

```
</> frontend/AppRoutes.jsx

<Routes>
  <Route path="/" element={user ? <Navigate to="/dashboard" /> : <Home />} />
  <Route path="/login" element={user ? <Navigate to="/dashboard" /> : <Login />} />
  <Route path="/register" element={user ? <Navigate to="/dashboard" /> : <Register />} />
  <Route
    path="/dashboard"
    element={
      <ProtectedRoute>
        <Dashboard />
      </ProtectedRoute>
    }
  />
  <Route
    path="/tracker"
    element={
      <ProtectedRoute>
        <MealTracker />
      </ProtectedRoute>
    }
  />
  <Route
    path="/profile"
    element={
      <ProtectedRoute>
        <Profile />
      </ProtectedRoute>
    }
  />
  <Route
    path="/admin/foods"
    element={
      <ProtectedRoute adminOnly={true}>
        <AdminFoods />
      </ProtectedRoute>
    }
  />
</Routes>
```

15. Security Considerations

Security was built into the project from the start rather than added on at the end. Here is a quick rundown of the main measures in place.

- **Passwords are never stored as plain text:** Every password is run through bcryptjs with a salt factor of 10 before it is saved. Even if someone got hold of the database, they would not find readable passwords.

- **JWT-based session management:** After logging in, users get a token that lasts 30 days. Every request to a protected endpoint must include this token — without it, the request is rejected immediately.
- **Admin actions are locked behind an extra check:** Even if a regular user somehow got a valid token, the admin middleware checks the `isAdmin` flag on their account before allowing any food management actions.
- **You can only modify your own data:** Meal and food update/delete endpoints verify that the logged-in user actually owns the record they are trying to change. You cannot edit someone else's meal.
- **All incoming data is validated:** `express-validator` checks the shape and content of every POST and PUT request. If something looks wrong, it is rejected with a clear error message before it touches the database.
- **CORS is configured explicitly:** The backend only accepts requests from known origins, preventing random websites from making API calls on a user's behalf.
- **Secrets stay out of the codebase:** The database connection string and JWT signing key live in a `.env` file that is excluded from version control. Nobody accidentally commits credentials to GitHub.

16. Conclusion

Building Nutrition Assistant was a genuinely useful exercise in putting together a real-world full-stack application — not just in terms of learning the MERN stack, but in thinking through what actually makes a product worth using.

The technical side came together well. The backend has a clean, secure REST API built on MVC conventions, with proper authentication, ownership checks, and input validation throughout. The frontend gives users something they can actually engage with — real-time macro breakdowns, goal-aware recommendations, and visual charts that make their weekly data easy to understand at a glance.

What made this project more interesting than a generic CRUD app was the Smart Assistant Hub and the pre-loaded Indian food database. Both came from thinking about who would actually use the app day-to-day and what would make it genuinely helpful for them, rather than just technically functional.

There is plenty of room to grow from here — a barcode scanner for packaged foods, social features to share progress, or a recommendation engine that learns from your habits over time. But as a foundation, the project does what it set out to do: make nutrition tracking simple, local, and a little bit smart.